

Amazon S3 — Stockage objet dans AWS

Objectifs

- expliquer à quoi sert Amazon S3 ;
- comprendre la différence entre stockage objet et système de fichiers ;
- expliquer les notions de **bucket**, **objet**, **clé** et **préfixe** ;
- créer et configurer un bucket S3 ;
- envoyer, télécharger, copier et supprimer des objets ;
- comprendre le caractère régional d'un bucket ;
- comprendre les mécanismes principaux de sécurité S3 ;
- comprendre le rôle d'IAM avec S3 ;
- expliquer le **Block Public Access** ;
- comprendre le chiffrement par défaut ;
- activer et utiliser le **versioning** ;
- comprendre les grandes classes de stockage ;
- utiliser S3 depuis la Console AWS et AWS CLI ;
- identifier ce qui peut générer des coûts ;
- nettoyer correctement les ressources utilisées.

1. Pourquoi avons-nous besoin d'un service comme S3 ?

Supposons qu'une application ait besoin de stocker :

- des images ;
- des vidéos ;
- des documents PDF ;
- des sauvegardes ;
- des fichiers CSV ;
- des logs ;
- des fichiers générés par les utilisateurs.

Une première idée serait de stocker ces fichiers directement sur le disque d'une machine EC2.

```
Utilisateur
|
v
Application EC2
|
v
Disque de l'instance
|
+-- image.jpg
+-- contrat.pdf
+-- sauvegarde.zip
```

1. Pourquoi avons-nous besoin d'un service comme S3 ?

Cela fonctionne, mais crée plusieurs problèmes.

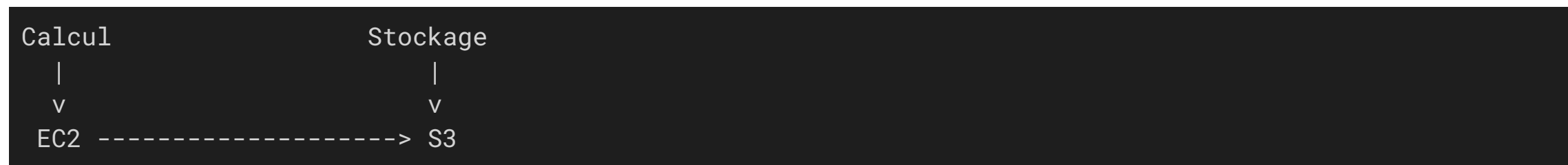
Que se passe-t-il si l'instance disparaît ?

Que se passe-t-il si plusieurs serveurs doivent accéder aux mêmes fichiers ?

Comment gérer plusieurs téraoctets ou pétaoctets de données ?

Comment conserver les fichiers indépendamment des machines qui les utilisent ?

Il est souvent préférable de séparer :



EC2 exécute l'application.

S3 conserve les données.

2. Qu'est-ce qu'Amazon S3 ?

Amazon Simple Storage Service, ou **Amazon S3**, est le service de **stockage objet** d'AWS. S3 permet de stocker des données sous forme d'objets et de les récupérer à travers les API AWS.

Exemples :

```
S3
|
+-- photo.jpg
+-- facture.pdf
+-- backup.zip
+-- donnees.csv
+-- video.mp4
```

S3 est utilisé notamment pour :

- stocker des fichiers applicatifs ;
- conserver des sauvegardes ;
- stocker des logs ;
- héberger les fichiers statiques d'applications web ;
- constituer des Data Lakes ;
- archiver des données ;
- distribuer des fichiers avec CloudFront ;
- stocker les fichiers produits ou consommés par EC2, Lambda et d'autres services AWS.

3. S3 n'est pas un disque dur distant

C'est probablement la distinction la plus importante à comprendre.

S3 n'est pas organisé comme un système de fichiers traditionnel.

Sur Linux, on travaille par exemple avec :

```
/home/alice/documents/rapport.pdf
```

avec :

- un disque ;
- des répertoires ;
- des sous-répertoires ;
- des fichiers.

S3 utilise un modèle différent :

```
Bucket  
|  
+-- Objet  
+-- Objet  
+-- Objet
```

Il s'agit de **stockage objet**.

4. Bucket et objet

Bucket

Un **bucket** est un conteneur S3.

Par exemple :

```
formation-s3-demo-829184
```

Il peut contenir des milliers ou des millions d'objets.

```
formation-s3-demo-829184  
|  
+-- logo.png  
+-- cours.pdf  
+-- utilisateurs.csv
```

Objet

Un **objet** correspond à une donnée stockée dans S3.

Un objet possède notamment :

- son contenu ;
- une clé ;
- des métadonnées ;
- une taille ;
- une classe de stockage ;
- éventuellement un identifiant de version.

Exemple :

```
Bucket  
formation-s3-demo-829184
```

```
Objet  
images/chat.jpg
```

5. La clé d'un objet

Chaque objet possède une **key**, ou clé.

Exemple :

```
images/chat.jpg
```

La clé identifie l'objet dans le bucket.

On peut donc représenter un objet par :

```
s3://formation-s3-demo-829184/images/chat.jpg
```

où :

```
formation-s3-demo-829184
```

```
|  
bucket
```

```
images/chat.jpg
```

```
|  
clé
```

6. Les « dossiers » S3

Dans la console S3, AWS affiche des dossiers :

```
images/  
documents/  
backups/
```

On pourrait donc imaginer :

```
bucket  
|  
+-- images/  
|   +-- logo.png  
|  
+-- documents/  
    +-- cours.pdf
```

Mais S3 ne possède pas réellement de répertoires comme Linux.

L'objet peut simplement avoir la clé :

```
images/logo.png
```

Le préfixe :

```
images/
```

permet à AWS de présenter cette structure comme un dossier.

On parle de **prefix**.

AWS précise que les préfixes permettent d'organiser les objets d'une manière similaire à des répertoires, mais qu'il ne s'agit pas réellement de répertoires.

7. Architecture générale

On peut représenter S3 simplement ainsi :

```
Compte AWS
|
v
Amazon S3
|
+-- Bucket A
|   |
|   +-- Objet 1
|   +-- Objet 2
|
+-- Bucket B
|   |
|   +-- Objet 3
```

Une application peut ensuite utiliser ces objets :

```
Application EC2
|
| API S3
v
Bucket S3
|
+-- images/logo.png
+-- documents/aide.pdf
```

8. S3 et les régions AWS

Lors de la création d'un bucket, on choisit une région AWS.

Par exemple :

```
Europe (Paris)
eu-west-3
```

ou :

```
Europe (Ireland)
eu-west-1
```

Le choix de la région peut dépendre :

- de la localisation des utilisateurs ;
- de la localisation des autres ressources AWS ;
- des contraintes réglementaires ;
- du prix ;
- de la latence.

AWS indique que les objets d'un bucket restent dans sa région tant qu'on ne configure pas explicitement leur transfert vers une autre région.

On peut donc avoir :

```
Region eu-west-3
|
+-- EC2
|
+-- S3 bucket
```

9. Le nom des buckets

Pour les buckets S3 classiques utilisant l'espace de noms partagé, le nom doit être unique parmi les comptes AWS de la partition AWS concernée.

Vous ne pourrez donc probablement pas créer :

```
test
```

si ce nom existe déjà.

On utilise plutôt quelque chose comme :

```
formation-s3-demo-583921
```

AWS impose également plusieurs règles, notamment :

- entre 3 et 63 caractères ;
- uniquement minuscules, chiffres, `.` et `-` ;
- début et fin par une lettre ou un chiffre.

AWS recommande également d'éviter les informations sensibles dans le nom d'un bucket.

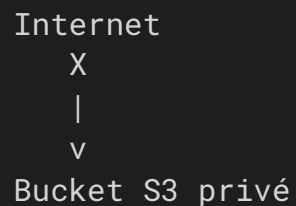
Pour le cours, nous utiliserons un **General purpose bucket** classique.

AWS propose désormais également un espace de noms régional réservé au compte.

10. Sécurité : un bucket doit-il être public ?

Non.

Dans la majorité des situations :



```
Internet
  X
  |
  v
Bucket S3 privé
```

Seules les identités autorisées doivent accéder aux données.

Par défaut, AWS active les quatre mécanismes de **S3 Block Public Access** sur les nouveaux buckets, et AWS recommande de les conserver sauf besoin explicite d'accès public.

Cela protège notamment contre une configuration accidentelle qui rendrait des fichiers accessibles à tout Internet.

11. IAM et S3

S3 ne remplace pas IAM.

IAM détermine notamment **qui peut effectuer quelles actions sur S3**.

Exemples d'actions :

```
s3:ListBucket
s3:GetObject
s3:PutObject
s3:DeleteObject
```

Par exemple :

```
Utilisateur IAM
|
| autorisé à effectuer s3:GetObject
v
Bucket S3
|
+-- rapport.pdf
```

Une identité sans permission recevra généralement :

```
AccessDenied
```

12. Bucket policy et IAM policy

Deux mécanismes importants peuvent intervenir.

Identity-based policy

Une policy IAM attachée à :

- un User ;
- un Group ;
- un Role.

Elle répond principalement à :

“ Que peut faire cette identité ?

”

Resource-based policy

S3 permet également d'attacher une **Bucket Policy** au bucket.

Elle répond principalement à :

“ Qui peut accéder à cette ressource et comment ? ”

Exemple conceptuel :

```
IAM Role
  |
  | identity policy
  v
Permissions
  |
  v
S3 bucket
  ^
  |
bucket policy
```

13. ACL

S3 possède historiquement un autre mécanisme appelé **ACL – Access Control List**.
Pour les nouveaux buckets classiques, AWS utilise par défaut :

```
Object Ownership:  
Bucket owner enforced
```

et les ACL sont désactivées.

Pour une formation moderne, il est préférable de privilégier :

```
IAM Policies  
+  
Bucket Policies  
+  
Block Public Access
```

plutôt que de commencer par les ACL.

14. Chiffrement

Les objets S3 sont aujourd'hui chiffrés côté serveur par défaut.

AWS applique au minimum :

```
SSE-S3
```

c'est-à-dire :

Server-Side Encryption with Amazon S3 managed keys.

Tous les nouveaux objets déposés dans S3 bénéficient automatiquement de ce chiffrement de base.

Architecture simplifiée :

```
Fichier
|
| upload
v
AWS S3
|
| chiffrement
v
Données chiffrées sur le stockage AWS
```

D'autres possibilités existent :

```
SSE-S3
SSE-KMS
DSSE-KMS
```

Pour debuter :

- **SSE-S3** : suffisant pour comprendre le chiffrement S3 ;
- **SSE-KMS** : à présenter plus tard lorsque KMS et la gestion des clés seront étudiés.

15. Versioning

Supposons que nous stockions :

```
rapport.txt
```

Puis que quelqu'un écrase accidentellement ce fichier.

Sans versioning :

```
rapport.txt V1
```

```
upload
```

```
rapport.txt V2
```

V1 est remplacée.

Avec **S3 Versioning** :

```
rapport.txt
```

```
|
```

```
+-- Version 3
```

```
+-- Version 2
```

```
+-- Version 1
```

S3 conserve plusieurs versions du même objet.

Le versioning est désactivé par défaut sur un nouveau bucket et peut être activé ensuite.

Cela permet notamment de récupérer des données après :

- un écrasement accidentel ;
- certaines suppressions accidentelles ;
- une erreur applicative.

16. Suppression dans un bucket versionné

Le comportement est intéressant.

Dans un bucket sans versioning :

```
DELETE rapport.txt
```

supprime l'objet.

Dans un bucket avec versioning, une suppression classique crée généralement un :

```
Delete Marker
```

Schématiquement :

```
rapport.txt  
|  
+-- Delete Marker      <- version courante  
+-- Version 2  
+-- Version 1
```

Le fichier semble supprimé, mais les anciennes versions existent encore.

Cela a également une conséquence importante :

“ les anciennes versions occupent toujours du stockage et peuvent donc continuer à générer des coûts. ”

17. Classes de stockage

Tous les fichiers ne sont pas utilisés de la même façon.

Un logo de site web peut être consulté constamment.

Une sauvegarde vieille de cinq ans peut n'être consultée qu'en cas de catastrophe.

S3 propose donc différentes **storage classes**.

Quelques classes importantes :

```
S3 Standard  
S3 Intelligent-Tiering  
S3 Standard-IA  
S3 One Zone-IA  
S3 Glacier Instant Retrieval  
S3 Glacier Flexible Retrieval  
S3 Glacier Deep Archive
```

S3 Standard

Adapté aux données fréquemment utilisées.

Exemple :

```
images d'une application  
documents actifs  
fichiers téléchargés régulièrement
```

C'est une bonne classe pour débuter.

S3 Intelligent-Tiering

Intéressant lorsque le comportement d'accès aux données est difficile à prévoir.

S3 surveille l'utilisation et peut déplacer automatiquement les objets entre différents niveaux de stockage.

Les objets non consultés pendant 30 jours peuvent notamment passer dans un niveau d'accès moins fréquent, puis dans Archive Instant Access après 90 jours.

Classes Glacier

Utilisées pour les données rarement consultées et l'archivage.

Plus on accepte des contraintes concernant l'accès et la récupération, plus le stockage peut devenir économique.

Pour une introduction à AWS, il suffit de retenir :

Accès fréquent	----->	Archivage
S3 Standard		Glacier
plus immédiatement accessible		optimisé archivage

Les détails de tarification et de durée minimale de stockage devront être étudiés avant de choisir une classe pour une vraie architecture.

18. Cycle de vie — Lifecycle

Imaginons des sauvegardes :

```
backup-j1.zip  
backup-j2.zip  
backup-j3.zip  
...
```

On pourrait décider :

```
0 jour      -> S3 Standard  
30 jours    -> classe moins coûteuse  
365 jours   -> suppression
```

S3 permet d'automatiser ce comportement grâce aux **Lifecycle Rules**.

Cela évite de devoir écrire un programme chargé de déplacer ou supprimer manuellement les objets.

19. Coûts S3

Un bucket vide n'est pas le principal problème.

Les coûts S3 peuvent notamment dépendre :

```
stockage  
+  
requêtes  
+  
récupérations selon la classe  
+  
transferts de données  
+  
certaines fonctionnalités supplémentaires
```

AWS facture S3 à l'utilisation et ne se limite donc pas au simple nombre de gigaoctets stockés.

Exemple

Si une application contient :

```
1 000 000 fichiers
```

il faut réfléchir non seulement à :

```
combien de Go ?
```

mais aussi :

```
combien de PUT ?  
combien de GET ?  
combien de données transférées ?  
quelle Storage Class ?
```

20. Attention au Free Tier

Il ne faut plus enseigner simplement :

“ « S3 est gratuit jusqu'à X Go pendant 12 mois » ”
comme une règle générale valable pour tous les nouveaux comptes.

Le programme AWS Free Tier a évolué.

Pour les nouveaux clients concernés depuis le **15 juillet 2025**, AWS propose notamment jusqu'à **200 USD de crédits**, applicables à des services éligibles comme S3, avec une offre gratuite disponible pendant six mois selon les conditions du programme.

Il faut donc toujours vérifier :

AWS Billing
AWS Free Tier
AWS Budgets

21. Bonnes pratiques essentielles

Pour une première utilisation de S3 :

- conserver **Block Public Access** activé ;
- utiliser IAM pour contrôler les accès ;
- privilégier les rôles IAM pour les applications AWS ;
- éviter les credentials permanents dans le code ;
- conserver les ACL désactivées sauf besoin spécifique ;
- choisir consciemment la région ;
- activer le versioning lorsqu'il apporte une protection utile ;
- surveiller les anciennes versions ;
- choisir une classe de stockage adaptée ;
- utiliser les Lifecycle Rules lorsque cela permet d'automatiser la conservation ;
- surveiller Billing et Budgets ;
- supprimer les ressources pédagogiques inutiles.

22. À retenir

S3 = stockage objet

Les données sont organisées en :

```
Bucket
|
+-- Object
    |
    +-- Key
    +-- Data
    +-- Metadata
```

Un chemin comme :

```
documents/2026/rapport.pdf
```

est principalement une **clé** utilisant des préfixes.

Un bucket est créé dans une région AWS.

S3 est privé par défaut.

IAM contrôle qui peut effectuer des opérations.

Block Public Access aide à empêcher une exposition accidentelle.

Les nouveaux objets S3 sont chiffrés par défaut.

Le versioning permet de conserver plusieurs versions d'un objet.

Les Storage Classes permettent d'adapter stockage et coût au type d'utilisation.

Cheat sheet S3 CLI

Lister les buckets :

```
aws s3 ls --profile formation
```

Créer un bucket :

```
aws s3 mb s3://nom-unique-du-bucket --region eu-west-3 --profile formation
```

Lister un bucket :

```
aws s3 ls s3://nom-du-bucket --profile formation
```

Cheat sheet S3 CLI

Lister récursivement :

```
aws s3 ls s3://nom-du-bucket --recursive --profile formation
```

Envoyer un fichier :

```
aws s3 cp fichier.txt s3://nom-du-bucket/fichier.txt --profile formation
```

Télécharger :

```
aws s3 cp s3://nom-du-bucket/fichier.txt fichier.txt --profile formation
```

Copier un objet :

```
aws s3 cp s3://nom-du-bucket/source.txt s3://nom-du-bucket/copie.txt --profile formation
```

Cheat sheet S3 CLI

Supprimer un objet :

```
aws s3 rm s3://nom-du-bucket/fichier.txt --profile formation
```

Synchroniser un dossier :

```
aws s3 sync dossier-local s3://nom-du-bucket/prefixe --profile formation
```

Supprimer récursivement les objets visibles :

```
aws s3 rm s3://nom-du-bucket --recursive --profile formation
```

Supprimer un bucket vide :

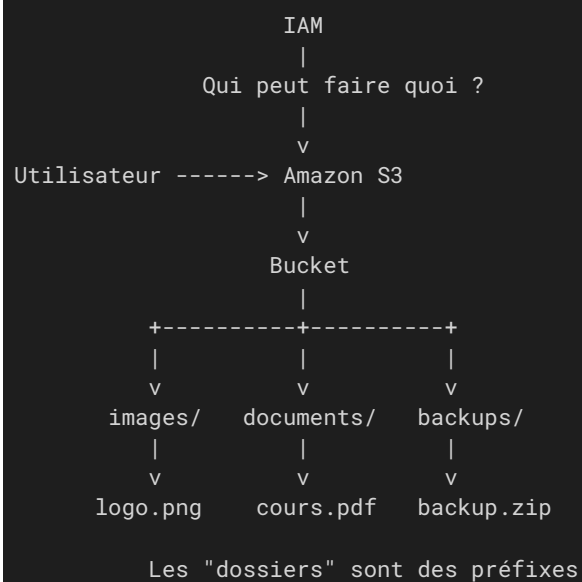
```
aws s3 rb s3://nom-du-bucket --profile formation
```

Cheat sheet S3 CLI

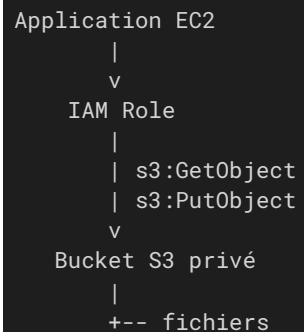
Vérifier l'identité utilisée :

```
aws sts get-caller-identity --profile formation
```

Schéma récapitulatif



Avec une application AWS :



Liens utiles

Documentation officielle Amazon S3 : [Guide utilisateur Amazon S3](#)

Bonnes pratiques de sécurité : [Bonnes pratiques de sécurité S3](#)

Tarification : [Tarification Amazon S3](#)

Référence AWS CLI : [AWS CLI – commandes S3](#)